Architecture des Ordinateurs

Michel Aillerie & Maxime Agius

Représentation binaire des informations.
Les codes minimum

Représentation binaire des informations. Les codes minimum

1. INTRODUCTION

Un mot mémoire est composé d'un certain nombre de digits binaires. Le binaire est la seule forme sous laquelle l'information peut être stockée.

Par exemple, un mot de 8 bits contient 256_D combinaisons différentes de 0 et de 1.

Dans le cas général, soit n = le nombre de digits de la représentation

Si la longueur du code, n est minimum, alors le code est un code minimum

La combinaison de 0 et de 1 donne lieu à une interprétation :

1) données numériques purement binaire indépendantes.

2) données numériques binaires interprétées ou faisant partie d'un nombre sur plusieurs mots.

3) code de données, c'est à dire combinaison de bit assujettis à une certaine règle.

4) code instruction, c'est à dire combinaison de bit devant être transmise à l'unité centrale.

Représentation binaire des informations. Les codes minimum

1. INTRODUCTION

Si l’on prend un mot mémoire quelconque, il est impossible d’interpréter son contenu. En fonction du séquencement des opérations l’ordinateur, lui, le sait. Par exemple, lorsque l'information est rangée en mémoire dans la zone réservée au programme, il est fort probable que cette information fasse partie d'une instruction, soit en tant que code opération, soit en tant que opérande ou adresse d'opérande. Par contre si cette donnée ce trouve dans la zone mémoire réservée aux données, on est sur que cette information est une variable (ou fait partie d'une variable multi-octet), correspondant à une variable binaire ou à une donnée interprétée.

Représentation binaire des informations. Les codes minimum

1. INTRODUCTION

- On ne fait dessus aucune interprétation. Sur papier, il est possible de représenter les nombres soit en binaire soit en octal, soit en Hexa.
- **<u>Exemple :</u>**

4 E	Hexa
01001110	Binaire
1 1 6	Octal

Ce n'est juste qu'une représentation différente.

- Pourquoi interpréter les informations binaires:

  Sur 8 bits, on peut coder une grandeur numérique < 255_D.

- Pour étendre la représentation des nombres, il est alors nécessaire d'effectuer une interprétation des nombres écrits en binaire pur.

  Par exemple, sur 16 bits on code un nombre < 65535. Mais rien indique dans cette écriture la façon dont les 2 octets composant ce mot de 16 bits sera rangé en mémoire.

Représentation binaire des informations. Les codes minimum

1. INTRODUCTION

Les mots de 8 bits peuvent être interprétés par pair et ainsi deux octets consécutifs en mémoire formeront la moitié inférieure, appelée les poids faibles et la moitié supérieure, appelée les poids forts d'un nombre de 16 bits.

Il est important de savoir lorsque l'on utilise directement les informations dans la mémoire d'un ordinateur si le processeur range en premier les poids faibles ou forts, vers les adresses décroissantes ou croissantes.

NOMB	2^{15} OCTET de POIDS FORT 2^{8}	2^{7} OCTET de POIDS faibles 2^{0}

	-
	-
+1	2^{15} OCTET de POIDS FORT 2^{8}
NOMB	2^{7} OCTET de POIDS faibles 2^{0}
	-
	-

Grâce à cette possible interprétation des nombres multi-octets, il est possible de coder un nombre sur 6 octets par exemple voir plus.

Représentation binaire des informations. Les codes minimum

1. INTRODUCTION

Conséquence sur la disposition des différents champs en mémoire

- **Complément à la base**

Le complément à la base d'un nombre correspond au complément à la base du premier digit de poids faible différent de zéro et complément à la base moins 1 (cpl logique) des digits de poids Fort.

	Octet 1	Octet 0
NOMB	**10110001**	**10010100**
cpl logique de NOMB	+ 01001110	01101011
+ 1		+ 1
cpl à B de NOMB	**01001110**	**01101100**

Représentation binaire des informations. Les codes minimum

2. ARITHMÉTIQUE DES NOMBRES MULTIOCTETS ET MULTIMOTS

- Le calcul à la main ou sur papier ne pose pas de problème ; c'est ce que l'on a vu précédemment. Pour l'ordinateur : il y aura le nombre de pas de calcul correspondant au nombre de mots sur lequel est codé le chiffre.
- Par exemple :

	Octet 1	Octet 0
	10110001	10000011
	+ 00011110	10100100
Retenue du mot 0 sur le mot 1	+ 1	
	11010000	00100111

- **Remarque :**

Lors de la complémentation à 2 il faut bien s'assurer de la position de l'octet de poids faibles pour le complémenter à la base et prendre le complément logique des octets de poids forts (cf. § cpl).

Si cette précaution n'était pas rigoureusement appliquée, le +1 du complément à la base par rapport au complément logique serait rajouté à chaque octet .

Représentation binaire des informations. Les codes minimum

3. NOMBRES BINAIRES SIGNÉS

3.1. CONVENTION 1

Un nombre négatif en binaire est représenté en complément à à la base.

3.2. CONVENTION 2

Pour définir le signe du nombre, N on réserve le bit de poids Fort comme bit de signe. Il devient le **Sign Flag, SF**:

Si SF = 0,
    alors N > 0
    sinon N < 0
fsi

- **Remarque: Conversion d'un nombre négatif d'une base B vers une base B'**

Si N <0, il est représenté par $(10^n\text{-}N)_B$.

Pour un changement de base, il faut faire la conversion de $-N_B$ en $-N_{B'}$ et non de $(10^n{}_B\text{-}N)_{B'}$.

Donc si N<0 ➔ Il faut représenter N en valeur absolue, soit $-|N_B|$, par complémentation à la base.

Seulement ensuite, on peut le convertir.

Représentation binaire des informations. Les codes minimum

3. NOMBRES BINAIRES SIGNÉS

- **Remarque 2 N** exprimés sur n digits binaires,

Avec des **nombres non signés**,

l'étendue de la représentation est: $0 \leq N_B < B^n$

Avec des **nombres signés**,

l'étendue de la représentation est: $-B^{n-1} \leq N_B < B^{n-1}$

- **Exemple N** exprimés sur 8 digits binaires

binaire	décimal
11111111	255
-	-
10000000	128
01111111	127
-	-
-	-
00000011	3
00000010	2
00000001	1
00000000	0

binaire	décimal
01111111	+127
-	-
-	-
00000011	3
00000010	2
00000001	1
00000000	0
11111111	-1
11111110	-2
11111101	-3
-	-
-	-
10000001	-127
10000000	-128

Représentation binaire des informations. Les codes minimum

4. NOMBRES DÉCIMAUX CODÉS EN BINAIRE

Sur 4 digits binaires, on peut coder 16 chiffres de 0 à F_H soit, de 0 à 15_D.

Si l'on ignore dans le codage les combinaisons > à 9, on peut alors coder des nombres décimaux.

Lorsque les 9 premières combinaisons sont utilisées, on a alors le décimal codé binaire ou **le BINARY CODED DECIMAL (BCD)**.

Exemple : 359D = $0011\ 0101\ 1001_{BCD}$

Il est important de bien se rendre compte que pour coder un chiffre en décimal , on a bien besoin des 4 bits pour coder de 0 à 9 même si **6 positions sont inutilisées**.

Suite à ce codage,
la **base de numération reste le décimal**
même si
la **base de représentation est le binaire**.

Décimal	Binaire
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
~~10~~	~~1010~~
~~11~~	~~1011~~
~~12~~	~~1100~~
~~13~~	~~1101~~
~~14~~	~~1110~~
~~15~~	~~1111~~

Représentation binaire des informations. Les codes minimum

4. NOMBRES DÉCIMAUX CODÉS EN BINAIRE

4.1. L'addition de nombres BCD

Le principe est basé sur le fait que normalement, en décimal, une retenue sera générée lorsque le résultat de l'addition de 2 nombres sera >9.

En binaire sur 4 digits, une retenue sera générée lorsque le résultat sera $>15_D$

La différence est donc de 6 entre la même opération effectuée dans ces 2 bases.

En BCD, les calculs s'effectuant en binaire, on rajoutera 6 lorsque le résultat passe au delà de 9.

- Basé sur ce principe, il existe deux méthodes pour l'addition RES = N1 + N2:

  - **L'addition classique**:

Dans le résultat, pour chaque chiffre > $9_D = 1010_B$, ou si une retenue apparaît , on rajoute alors 6.

  - **L'addition avec retenue anticipée**:

On imagine que le résultat en décimal serait passé au delà de 9. Alors, on ajoute 6 à chaque chiffre de N1. Pour chaque chiffre du résultat, si effectivement une retenue est générée, alors c'est que l'opération sur ces 2 chiffres est passé au delà de 9, sinon, on retranche le 6 en trop.

Représentation binaire des informations. Les codes minimum

4. NOMBRES DÉCIMAUX CODÉS EN BINAIRE

4.1. L'addition de nombres BCD Exemple 29 + 34 = 63

L'addition classique:

29	0010 1001	
34	0011 0100	
63	0101 1101	retenue sur poids faibles
	0000 0110	On rajoute 06_H
63 =	0110 0011	

L'addition avec retenue anticipée:

29_D		0010 1001	
	+66	0110 0110	
	pas 1	1000 1111	
$+34_D$		0011 0100	
	pas 2	1100 0011	
	retenues	0 1	
	Facteur	1010 0000	$=A0_H$
$=63_D$		0110 0011	$= 63_H$
	retenue du pas 2	0	

4. NOMBRES DÉCIMAUX CODÉS EN BINAIRE

4.2. La soustraction de nombres BCD

Les opérations en BCD se font en binaire et la soustraction de 2 chiffres, soit sur 4 digits binaires, N1-N2 en BCD s'effectue par N1 + (10^4_B - N2).

Mais N1 et N2 sont des nombres décimaux. Le cplt aurait du être calculé en décimal, or:

$$(10^4_B - N2) = (10_H - N2) =(10_D+6_D - N2).$$

On voit que l'on a rajouté implicitement 6 par rapport au complément en décimal.

Si l'on compare avec l'addition en BCD, on s'aperçoit que l'on a effectué le pas 2 de l'addition avec retenue anticipée.

Par conséquent il suffit de continuer par le pas 3 décrit pour l'addition avec retenue anticipée en , méthode machine:

Pour chaque chiffre du résultat, si effectivement une retenue est générée, alors c'est que l'opération sur ces 2 chiffres est passé au delà de 9, sinon, on retranche le 6 en trop

Représentation binaire des informations. Les codes minimum

4. NOMBRES DÉCIMAUX CODÉS EN BINAIRE

4.2. La soustraction de nombres BCD: Exemple1: 71-25 = 46

25_D		0010 0101	
-25_D	pas 1	1101 1011	cpl à 2
$+71_D$		0111 0001	
	pas 2	0100 1100	
	retenues	1 0	CF/AF
	Facteur	1111 1010	$-06_H = FA_H$
		1 0100 0110	$= 46_H$

La vraie retenue=1
➔Res>0

Table des facteurs additionels

CF	AF	Facteur additionel
0	0	$-66_H = +9A_H$
0	1	$-60_H = +A0_H$
1	0	$-06_H = +FA_H$
1	1	00

Représentation binaire des informations. Les codes minimum

4. NOMBRES DÉCIMAUX CODÉS EN BINAIRE

4.2. La soustraction de nombres BCD: Exemple 2: 25 - 71 = -46

71_D			0111 0001	
-71_D	pas 1		1000 1111	cpl à 2
$+25_D$			0010 0101	
	pas 2		1011 0100	
	retenues	0	1	CF/AF
	Facteur		1010 0000	$-60_H = A0_H$
		1	0101 0100	$= 54_H$

La vraie retenue=0
➔Res<0

La retenue du pas 2 dans cet exemple est 0. Le résultat 54 représente donc en réalité -(100-54) soit -46

54	0101 0100	
- 54	1010 1100	cpl à 2
$+9A_H$	1001 1010	
	0100 0110	$= 46_H$

Représentation binaire des informations. Les codes minimum

4. NOMBRES DÉCIMAUX CODÉS EN BINAIRE

4.3. Procédure de conversion de nombre BCD en binaire. Exemple d'un Nb fractionnaire

- On frappe les chiffres au clavier que l'on va ranger dans un tableau TAB: Soit N = $0{,}36_D$

+3	0000 0000
+2	0000 0000
+1	0000 0110
TAB (0)	0000 0011

Solution manuelle (calcul en décimal)

N *2 = 0,36 x 2 = **0**,72
0,72 x 2 = **1**,44
0,44 x 2 = **0**,88
0,88 x 2 = **1**,76
0,76 x 2 = **1**,52 ...etc....
soit N = $0{,}36_D$ = 0,01011…B

Solution machine (calcul en binaire)

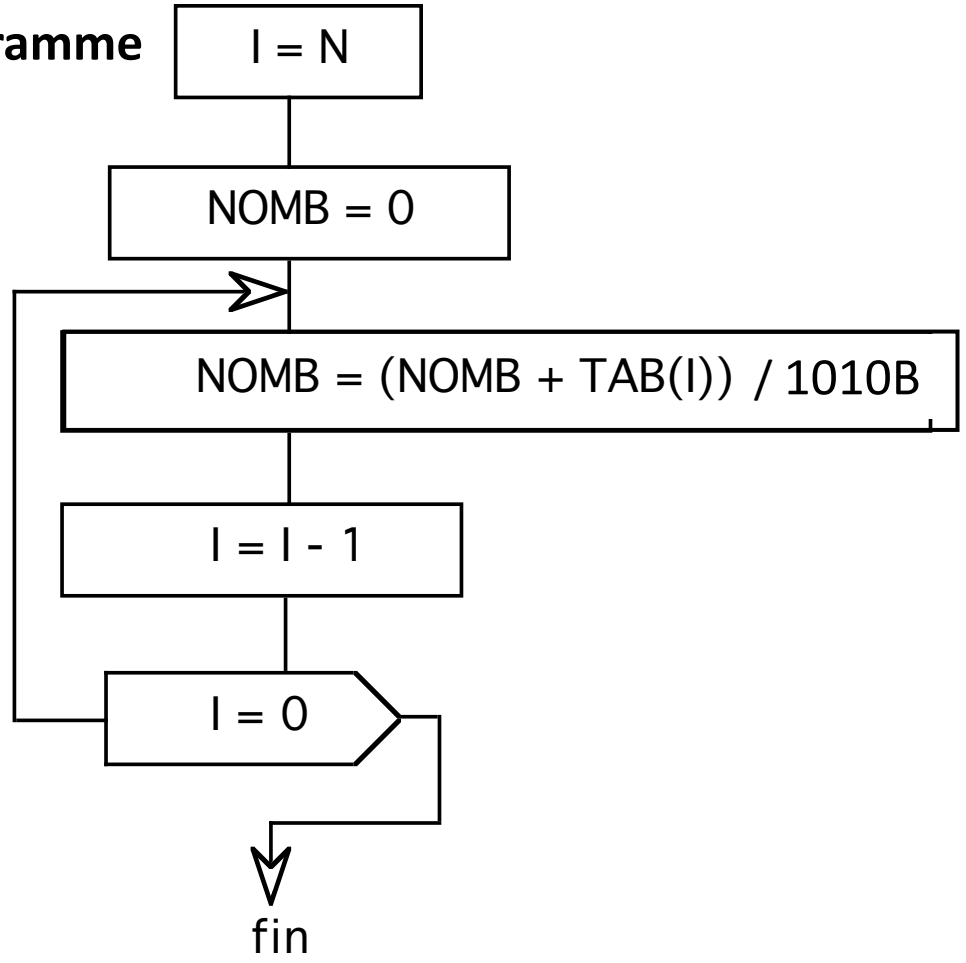
$0{,}36 = ((6 \times 10^{-1}) + 3) \times 10^{-1}{}_D$
$= ((0110/1010) + 0011\ ) / 1010_B$
0110/1010 = 0,100110011...
0,100110011...+ 11 = 11,100110011...
11,100110011... / 1010 = 0,010111…

Représentation binaire des informations. Les codes minimum

4. NOMBRES DÉCIMAUX CODÉS EN BINAIRE

4.3. Procédure de conversion de nombre BCD en binaire (Calculs faits en binaire)

Organigramme



Algorithmes

<u>Boucle Tant que</u>

I = 0
NOMB = 0
<u>Tant que</u> I < N <u>faire</u>
    NOMB := (NOMB + TAB (I))/1010B
    I := I + 1
<u>Ftq</u>

<u>Boucle répéter</u>

I = N
NOMB = 0
<u>Répéter jusqu'à</u> I = N, pas -1
    NOMB = (NOMB + TAB(I))/1010B
<u>Fin Répéter</u>

Représentation binaire des informations. Les codes minimum

5. LA REPRÉSENTATION DES GRANDS NOMBRES

En astronomie, l'étendue des nombres utilisés va de 0 à 10^{30}. En chimie, elle va de 0 à 10^{-30}. La physique utilise toutes les nombres appartenant à l'intervalle 10^{-30} - - -10^{30}. En mathématique, 10^{-oo} - - -10^{+oo}.

Si l'on ne dispose que de l'écriture au format fixe comme précédemment et si l'on veut toute la précision sur le nombre, il faut l'écrire en entier.

2 3 4 5 6 7 9 8 7 4 3 5 2 1 ----------3 2 1 0.

Mais en fonction de la précision que l'on désire ou de la précision des calculs effectués sur le nombre, juste quelques digits peuvent servir.

2 3 4 5 6 0 0 0 ----------0 0 0 0 (ici les 5 premiers chiffres de poids Forts)

Il y a une perte de qualité du codage car l'information utile par rapport à l'information totale possible est très petite.

De plus des erreurs peuvent intervenir rapidement du fait de la difficulté du comptage des zéros.

=>

La solution est d'utiliser un codage en **notation scientifique** avec mantisse et exposant. C'est la **représentation en virgule flottante**.

5. LA REPRÉSENTATION DES GRANDS NOMBRES

- **Conversion d'un nombre écrit en VFN vers le décimal**

Soit $N = 0.11010101_B * 10_B^{01010001B}$

- Conversion de la mantisse: $M = 0.11010101_B = 2^{-1} + 2^{-2} + 2^{-4} + 2^{-6} + 2^{-8} = 0{,}83203125_D$
- Conversion de l'exposant: $E = 01010001_B = 2^6 + 2^4 + 2^0 = 81_D$
- Changement de base de représentation de l'exposant

$$2^x = 10^y \rightarrow \log 2^x = \log 10^y \rightarrow x \log 2 = y \log 10 \rightarrow y = x (\log 2 / \log 10) = 0{,}30103\ x$$

Dans notre exemple

- $2^{81} = 10^{(81*0{,}30103)} = 10^{24{,}38343} = 10^{24\ +\ 038343} = 10^{0{,}38343} \times 10^{24} = 2{,}4178536 \times 10^{24}$
- D'où le nombre:
- $N = (0{,}83203125 \times 2{,}4178536) \times 10^{24} = 2{,}0117296.\ 10^{24}{}_D$

Une représentation possible dite normalisée (voir § suivants) consiste à mettre la virgule décimale juste avant le 1° digit significatif.

soit $N = 0{,}20117296.\ 10^{25}{}_D$

Le procédé inverse est utilisé pour avoir la conversion du décimal en binaire.

5. LA REPRÉSENTATION DES GRANDS NOMBRES

- **La représentation en virgule flottante**

La **représentation en virgule flottante** consiste à représenter les nombres sous la forme:

$$N = M*B^E$$

avec M = la mantisse, B= la base et E = l'exposant,

- **Exemple**

$$N = 0{,}00000001=1*10^{-8}=0{,}1*10^{-7}=10*10^{-9}$$

- **Remarque 1**

La représentation des nombres en virgule flottante, pour une manipulation à l'aide de moyens informatiques nécessite certaines normalisations quant aux formats et bases utilisées pour la représentation des différentes parties. On obtient alors la représentation en **Virgule Flottante Normalisée ou VFN**.

- **Remarque 2**

La taille de la mantisse M, défini le maximum de chiffres significatifs (la précision du codage).

- La taille de l'exposant E, défini l'amplitude (la dynamique) du codage.

5. LA REPRÉSENTATION DES GRANDS NOMBRES. VFN

- **La représentation en Virgule Flottante Normalisée**

Il n'y a pas une représentation unique en VFN. Les constructeurs ont presque tous définis leur représentation en VFN.

Les conventions à définir pour représenter un nombre en VFN sont:

- La base B du codage de la mantisse (en général une base binaire pour sa simplicité machine;),
- La base de la représentation de la mantisse et de l'exposant (c'est bien sûr le binaire),
- Le nombre de digits, m réservé dans la représentation à la mantisse (précision du codage),
- Le nombre de digits, e réservé dans la représentation à l'exposant (dynamique du codage),
- La convention utilisée pour le signe de l'exposant,
- La convention utilisée pour le signe du nombre,
- La disposition des différents champs,
- La représentation du zéro.

5. LA REPRÉSENTATION DES GRANDS NOMBRES. VFN

- **La représentation de l'exposant**

2 conventions sont en général utilisées:

1- Le complément à 2 c'est à dire $\mathbf{-2^{e-1} \leq E \leq 2^{e-1}-1}$

2- Le complément excentré pour le rendre toujours positif. Il est appelé alors **caractéristique**, C, c'est à dire que l'on pose $C = E+2^{e-1}$ soit $\mathbf{0 \leq C \leq 2^{e-1}}$

Dans ce cas avec un caractéristique, il n'y a pas de manipulation directe d'exposant négatif.

- **Exemple 1:**

Caractéristique sur 8 bits: $2^e-1 = 128$ ➔ $C = E+128$ soit $0 \leq C \leq 255$

- **Exemple 2:**

Caractéristique sur 11 bits: $2^{e-1} = 1024$ ➔ $C = E+1024$ soit $0 \leq C \leq 2047$

- **Remarque:**

Dans certaines normes, telle l'IEEE 754, le décalage de la mantisse par rapport à l'exposant n' est pas effectué avec la valeur 2^{e-1}. Ceci permet de réserver quelques positions spéciales en cas de problème de codage.(sous dépassement de capacité, dépassement de capacité...)

5. LA REPRÉSENTATION DES GRANDS NOMBRES. VFN

- **La représentation de la mantisse**

Pour obtenir le maximum de chiffres significatifs de la mantisse, il faut normaliser le nombre en le représentant avec un digit à droite ou à gauche de la virgule diffèrent de zéro.

- **Exemple**

$$N = 11{,}375_D = 1011{,}011_B = 0{,}1011011_B * 2^4 = 1{,}011011_B * 2^3$$

Dans le cas de la représentation binaire, ce nombre étant toujours à 1.

- **Optimisation supplémentaire**

Dans le cas du binaire, c-à-d VFN_2, certaines normes ne font pas apparaître le digit à 1 à droite ou à gauche (en fonction de la norme) dans l'écriture.

Ce bit caché dans la représentation (mais pas dans la valeur de la mantisse) est nommé le **bit caché** ou **hidden bit**.

La nouvelle mantisse dans l'exemple devient alors 011011_B.

5. LA REPRÉSENTATION DES GRANDS NOMBRES. VFN

- **La convention utilisée pour le signe du nombre**

3 conventions sont utilisées:

  - Valeur absolue de la mantisse et un bit de signe,
  - Complément à la base (binaire) de la mantisse seule,
  - Complément à la base de la mantisse et de l'exposant (ou caractéristique).et un bit de signe.

- **La disposition des différents champs**

  Elle est fonction des impératifs des constructeurs

- **La représentation du zéro**

C'est une exception à la normalisation qui veut un digit diffèrent de zéro à droite ou à gauche de la virgule.

2 conventions sont utilisées:

  - le plus petit réel codable
  - tous les digits à zéro

Représentation binaire des informations. Les codes minimum

5. LA REPRÉSENTATION DES GRANDS NOMBRES. VFN

- **Représentation type ICL**

```
 S  exposant  mantisse ( F est représenté)
|_|_________|___________________________|
  7          0 -2                      -24
```

Bit de signe + valeur absolue. La mantisse 0,1F. . (**avec bit caché)**.

N = S*0,1 . . . . . *2^{C-128}

Représentation binaire des informations. Les codes minimum

5. LA REPRÉSENTATION DES GRANDS NOMBRES. $VFN_{IEEE-754}$

- **Représentation norme IEEE-754:** trois représentations utilisées dans les microprocesseurs actuels

Simple précision: 32 bits (4 octets): 1 bit de signe, 8 bits d'exposant, 23 bits de mantisse
La mantisse s'exprime 1,F. . **(avec bit caché)**. **N = S*1,. . . . . *2^{C-127}**

```
 S    caract.     mantisse ( F est représenté)
|_|_________|_________________________|
   7          0 -1                          -23
```

Double précision: 64 bits (8 octets): 1 bit de signe, 11 bits d'exposant, 52 bits de mantisse

La mantisse s'exprime 1,F. . **(avec bit caché)**. **N = S*1,. . . . . *2^{C-1023}**

```
S  caractéristique                mantisse( F est représenté)
|_|_______________|_____________________________________________|
   10                0 -1                                          -52
```

Quadruple précision : 128 bits (16 octets) 1 bit de signe, 15 bits d'exposant, 112 bits de mantisse

Représentation binaire des informations. Les codes minimum

5. LA REPRÉSENTATION DES GRANDS NOMBRES. $VFN_{IEEE-754}$

Simple précision: 32 bits (4 octets): 1 bit de signe, 8 bits d'exposant, 23 bits de mantisse

La mantisse s'exprime 1,F. . **(avec bit caché)**. **N = S*1,. . . . . *2^{C-127}**

```
 S    caract.      mantisse (F est représenté)
|_|_________|___________________________|
   7          0 -1                          -23
```

- Exposant (E): de – 126 à 127
- Caractéristique C = E + 127 (et non +128 comme possible)
- Partie F représentée: de 0 à $1-2^{-23}$
- Mantisse: de 1 à $2-2^{-23}$
- Plus petit nombre normalisé: 2^{-126}
- Plus grand nombre normalisé: presque 2^{128}
- Les exposants 00000000 et 11111111 sont interdits, réserver pour indiquer: des erreurs (#NaN), le « 0 », les (sous-) dépassements de capacités.

Représentation binaire des informations. Les codes minimum

5. LA REPRÉSENTATION DES GRANDS NOMBRES. $VFN_{IEEE-754}$

- **Représentation de N = $3{,}1415910_D$ norme IEEE-754-SP:**

Bit de signe + valeur absolue. La mantisse s'exprime 1,F. . **(avec bit caché)**. **N = S*1,. . . . . *2^{C-127}**

Simple précision: sur 32 bits:

S caract. mantisse

|_|_____|_____|

7 0 -1 -23

1ère étape: Conversion en binaire (on peut passer par l'Hexa)

- Conversion de $3_D = 3_H$
- Conversion de $0{,}14159_D$ =
  - 0,14159 x16 = 2,26544 = **2** + 0,26544 ➔ $a'_{-1} = \mathbf{2}_H$
  - 0,26544 x 16 = 4,24704 = **4** + 0,24704 ➔ $a'_{-2} = \mathbf{4}_H$
  - 0,24704 x 16 = 3,95264 = **3** + 0,95264 ➔ $a'_{-3} = \mathbf{3}_H$
  - 0,95264 x16 = 15,24224 = **15** + 0,24224 ➔ $a'_{-4} = \mathbf{F}_H$
  - 0,24224 x 16 = 3,87584 = **3** + 0,87584 ➔ $a'_{-5} = \mathbf{3}_H$
  - 0,87584 x 16 = 14,01344 = **14** + 0,01344 ➔ $a'_{-6} = \mathbf{E}_H$

➔ $N = 3{,}1415910_D = 3{,}243F3E\ldots_H$

$N = 11{,}0010\ 0100\ 0011\ 1111\ 0011\ 1110\ldots_B$

$\mathbf{N = 1{,}10010010000111111001111 10\ldots_B \times 10^1{}_B}$

2ème étape: Calcul de la caractéristique: $\mathbf{C = E + 127_D = 1 + 127_D = 128_D = 10000000_B}$

3ème étape: Représentation de N.

S caract. mantisse

N = |**0**|**10000000**|**10010010000111111001111**| = **40 49 0F $CF_{VFN2-IEEE754-SP}$**

7 0 -1 -23

5. LA REPRÉSENTATION DES GRANDS NOMBRES. L'Arithmétique VFN

- **Normalisation utilisée dans ce paragraphe :**

Soit 2 nombres en VFN_2 :$N1 = M1 \times 2^{E1}$ et $N2 = M2 \times 2^{E2}$

- Mantisse sur 24 bits de la forme M = 0,F
- Exposant sur 8 bits .

- **La multiplication**

Le produit N = N1 x N2 s'écrit:

$$N = N1 \times N2 = (M1 * M2) \times 2^{(E1 + E2)}$$

Ceci induit une mantisse écrite sur 24 bits * 24 bits = 48 bits. Il est alors nécessaire de re-normaliser.

La partie intéressante de la mantisse à conserver correspond bien sur aux poids forts du résultat sur 48 bits ce qui nécessite, pour la normalisation un décalage à droite de 24 bits soit une multiplication de la mantisse par 2-24 et une compensation sur l'exposant par une multiplication par 2^{24} soit

$$N = [(M1 \times M2) * 2^{-24}] \times 2^{(E1 + E2 + 24)}$$

5. LA REPRÉSENTATION DES GRANDS NOMBRES. L'Arithmétique VFN

- **La division**

La division N = N1 ÷ N2 s'écrit:

$N = (M1 \div M2) \times 2^{(E1 - E2)}$ que l'on re-normalise par $N = ((M1 \div M2) \times 2^{24}) \times 2^{(E1 - E2-24)}$

- **L'addition et la soustraction :**

L'addition N = N1 ± N2 s'écrit $N = (M1 \times 2^{E1}) \pm (M2 \times 2^{E2})$

Plusieurs cas peuvent se présenter en fonction des valeurs des exposants:

Si E1 = E2 **alors** l'opération se déroule directement et sans problème particulier.

Sinon **Si** (par exemple) E1 > E2, il faut décaler la mantisse M2 de E1 – E2 digits.

Fsi

Fsi

$N = M1 \times 2^{E1} \pm M2 \times 2^{-(E1- E2)}$ $2^{E1} = (M1 \pm M2 * 2^{- (E1- E2)})* 2^{E1}$

- Dans tous les cas, il se peut que cela déborde tout de même. Il faut alors rattraper le débordement par un décalage à droite.

Représentation binaire des informations. Les codes minimum

6. Problèmes généraux de l'arithmétique binaire

- **Les indicateurs**

Dans l'unité de traitement, il existe un registre **Registre d'état** contenant les indicateurs, les **Flags**. (Autre noms: mot d'état du processeur ou Processeur Status Word (PSW)).

Les indicateurs sont positionnés

- par les résultats des calculs comme le **Zero Flag (ZF), Carry Flag (CF), Overflow Flag (OF), Sign Flag (SF), Auxiliary Flag (AF)**
- par l'utilisateur comme les masques d'interruptions: Interruption Flag (IF), comme les sens de parcours de l'adressage indexé: Direction Flag (DF).

Relation entre CF et OF

Exemple : n = 3

011	3
110	2
001	1
000	0
111	-1
110	-2
101	-3
100	-4

1°)
$$\begin{array}{r} 010 = 2 \\ +\ 110 = 1 \\ \hline 011 = 3 \end{array}$$
pas de d'overflow car pas de carry sur les bits codant la valeur absolue.

2°)
$$\begin{array}{r} 011 = 3 \\ +\ 001 = 1 \\ \hline 100 \neq 4 \end{array}$$
overflow

3°)
$$\begin{array}{r} 101 = -3 \\ +\ 111 = -1 \\ \hline 100 \neq -4 \end{array}$$
pas overflow mais c = 1

4°)
$$\begin{array}{r} 101 = -3 \\ +\ 110 = -2 \\ \hline 011 \neq -5 \end{array}$$
overflow + carry = 1

Représentation binaire des informations. Les codes minimum

7. LES CODES DE DONNÉES

- **LE CODE HOLLERITH**

Utilisé pour la perforation des cartes.

Code dont les mots sont de longueur 12 soit 4096 possibilités dont moins de 70 sont utilisées.

Une position binaire à 1 correspond à 1 perforation dans une carte.

Ce code à perdu de son utilité avec l'augmentation de la fiabilité des systèmes d'acquisition et de manipulation de l'information du en particulier, aux progrès de l'électronique ayant entraîné l'abandon des systèmes mécaniques tels que les lecteurs de cartes, perforateurs.

Représentation binaire des informations. Les codes minimum

7. LES CODES DE DONNÉES

- **LE CODE BAUDOT OU TÉLÉGRAPHIQUE INTERNATIONAL N°2**

Il a été conçu et employé pour le réseau télégraphique et téléphone, c'est à dire en transmission donc en séquentiel.

Mots de longueur 5 soit 32 combinaisons possibles ➔ insuffisant pour l'ensemble des lettres et des chiffres.
Principe pour pouvoir tout coder (sans distinction des majuscules et des minuscules):
2 caractères (29 et 30) permettent l'inversion des lettres et des chiffres permettant de définir dans quel sous-ensemble de 26 caractères se trouve la configuration. On a ainsi 58 configurations.

Ce n'est pas un code pondéré (à la construction du code, Baudot a tenu compte de la fréquence d'emploi des lettres et des chiffres). De ce fait, il n'est pas adapté aux traitements direct par ordinateur.

N°	Table haute	Table basse	Code		N°	Table	Table	Code
1	A	-	11000		17	Q	1	11101
2	B	?	10011		18	R	4	01010
3	C	:	01110		19	S	'	10100
4	D	*	10010		20	T	5	00001
5	E	3	10000		21	U	7	11100
6	F	(réservé)	10110		22	V	=	01111
7	G	(réservé)	01011		23	W	2	11001
8	H	(réservé)	00101		24	X	/	10111
9	I	8	01100		25	Y	6	10101
10	J	Appel	11010		26	Z	+	10001
11	K	(	11110		27	← Retour		00010
12	L	)	01001		28	≡ Interligne		01000
13	M	.	00111		29	↑ Inversion		11111
14	N	,	00110		30	↓ Inversion		11011
15	O	9	00011		31	Espace		00100
16	P	0	01101		32	(réservé)		00000

Représentation binaire des informations. Les codes minimum

7. LES CODES DE DONNÉES

- **LE CODE ASCII (AMERICAN STANDARD CODE FOR INFORMATION INTERCHANGE)**

Longueur 7 bits. C'est un code utilisé en transmission de données. (TTY - console – terminaux, Réseaux ...).
Il permet le codage des majuscules et des minuscules, des chiffres et des fonctions de commande.
Il a été conçu pour obtenir une protection dans les transmissions tout en permettant une conversion facile des chiffres et des lettres et un passage facile des majuscules aux minuscules et vis versa.
Par ailleurs, les colonnes intermédiaires constituent les chiffres + les majuscules soit un code ASCII simplifié.
Les 2 premières colonnes constituent des commandes pour la transmission de donnée.

Il y a possibilité de transmettre des combinaisons hors code grâce à 3 caractères spéciaux :

ESC modifie la signification du caractère qui suit.
SI SHIFT - IN: Retour au code.
SO SHIFT - OUT: Hors du code normalisé.

	000	001	002	003	004	005	006	007
0	NUL	DLE	SP	0	@	P	`	p
1	STX	DC1	!	1	A	Q	a	q
2	SOT	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
A	LF	SUB	*	:	J	Z	j	z
B	VT	ESC	+	;	K	[	k	{
C	FF	FS	,	<	L	\	l	|
D	CR	GS	–	=	M	]	m	}
E	SO	RS	.	>	N	^	n	~
F	SI	US	/	?	O	_	o	DEL

Représentation binaire des informations. Les codes minimum

7. LES CODES DE DONNÉES

- **L'UNICODE**

C'est un code développé suite au développement de l'internet: Il fallait développer un codage permettant la portabilité à l'international des applications (web) développées et la prise en compte des langues nationales.

Il a été développé en 1991 et en Mars 2020 dans sa version 13.0, 5390 nouveaux caractères ont été ajoutés.

Unicode et ISO/CEI 10646 acceptent plusieurs formes de transformation universelle pour représenter un point de code valide. Soit l'Universal Transformation Format (UTF): UTF-8 ; UTF-16 ; UTF-32.*

Le nombre après UTF représente le nombre minimal de bits des codets avec lesquels un point de code valide est représenté.

L'UTF-8, spécifié dans le RFC 362912, est le plus commun pour les applications Unix et Internet. Son codage de taille variable lui permet d'être en moyenne moins coûteux en occupation mémoire (pour les langues à alphabet latin). L'UTF-8 assure aussi, et c'est son principal avantage, une compatibilité avec les manipulations simples de chaînes en ASCII dans les langages de programmation.

* https://fr.wikipedia.org/wiki/Unicode. Consultation nov. 2020

Représentation binaire des informations. Les codes minimum

7. LES CODES DE DONNÉES

- **L'UNICODE**

C'est un code crée suite au développement de l'internet: Il fallait développer un codage permettant la portabilité à l'international des applications (web) développées et la prise en compte des langues nationales.

Il a été développé en 1991 et en Mars 2020 dans sa version 13.0, 5390 nouveaux caractères ont été ajoutés.

Le consortium est composé des entreprises Apple, HP, IBM, Sun, Microsoft,… et des entreprises de développement logiciels. Il est exigé comme standard dans de nombreux langages et applications comme XML, Java, LDAP,…

Unicode et ISO/CEI 10646 acceptent plusieurs formes de transformation universelle pour représenter un point de code valide. Soit l'Universal Transformation Format (UTF): UTF-8 ; UTF-16 ; UTF-32.*

Le nombre après UTF représente le nombre minimal de bits des codets avec lesquels un point de code valide est représenté.

* https://fr.wikipedia.org/wiki/Unicode. Consultation nov. 2020

Représentation binaire des informations. Les codes minimum

7. LES CODES DE DONNÉES

- **L'UNICODE UTF 8**

L'UTF-8, spécifié dans le RFC 362912, est le plus commun pour les applications Unix et Internet. Son codage de taille variable lui permet d'être en moyenne moins coûteux en occupation mémoire (pour les langues à alphabet latin). Codage initial sur 16 bits devant regrouper l'ensemble des alphabets latins, cyrillique, grec, hébreux, chinois, japonais,…

L'UTF-8 assure aussi, et c'est son principal avantage, une compatibilité avec les manipulations simples de chaînes en ASCII dans les langages de programmation.

Sans modification, un logiciel ou site Internet peut satisfaire les exigences des différentes plates-formes, langues et pays.

BINARY FORMAT OF BYTES IN SEQUENCE

1ST BYTE	2ND BYTE	3RD BYTE	4TH BYTE	NUMBER OF FREE BITS	MAXIMUM EXPRESSIBLE UNICODE VALUE
0xxxxxxx				7	007FH (127)
110xxxxx	10xxxxxx			(5+6)=11	07FFH (2047)
1110xxxx	10xxxxxx	10xxxxxx		(4+6+6)=16	FFFFH (65535)
11110xxx	10xxxxxx	10xxxxxx	10xxxxxx	(3+6+6+6)=21	10FFFFH (1,114,111)

Représentation binaire des informations. Les codes minimum

7. LES CODES DE DONNÉES

- **L'UNICODE UTF 8**

Types compatibles avec Unicode dans les langages de programmation

Langage de programmation	Type pour un seul caractère	Type pour tout texte
C	`char[4]` [a] ou `wchar_t[2]` [b]	`char[]` ou `wchar_t[]`
C++	`char[4]` [a] ou `wchar_t[2]` [a]	`char[]` ou `wchar_t[]` ou `std::string` ou `std::wstring`
Java	`char[2]` ou `int` [c]	`char[]` ou `String`
Bibliothèque ICU (pour C/C++ ou Java)	`UChar`	`UChar[]` ou `String`, `UnicodeString`
JavaScript ou ECMAScript	`char` [d]	`string`
C# ou J#	`char`	`string`
Delphi	`char[4]` [a] ou `widechar[2]`	`string` [a] ou `widestring`
Python 2		`unicode`
Python 3		`str`

* https://fr.wikipedia.org/wiki/Unicode. Consultation nov. 2020

Représentation binaire des informations. Les codes minimum

7. LES CODES DE DONNÉES

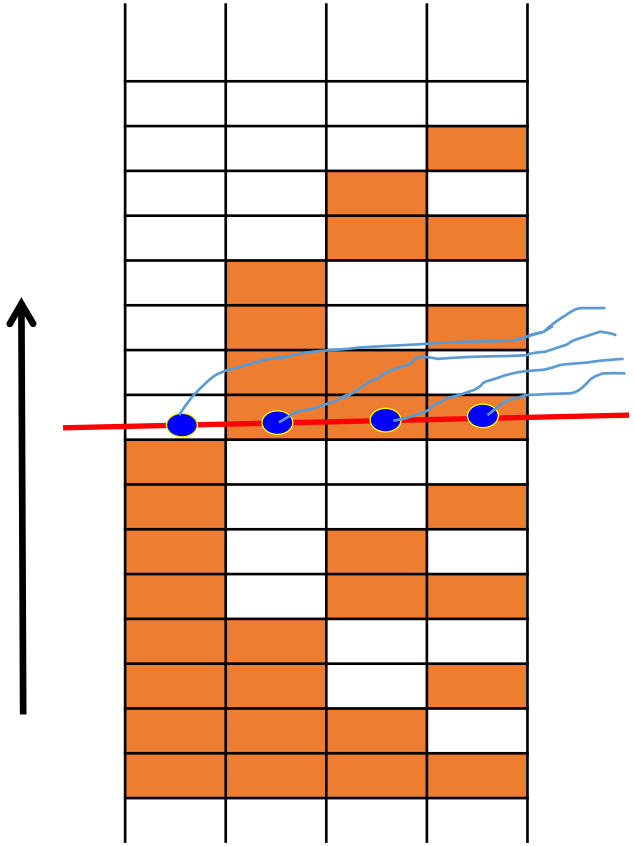
- **LES CODES GRAY ET CODES BINAIRES RÉFLÉCHIS**

Ils sont caractérisés par le fait que chaque élément du code se déduit du précédent par changement d'un seul digit.

Intérêt: Supposons un système devant passer de 0111 à 1000,

on a lors du passage entre n'importe qu'elle valeur en binaire pur.

- Exemple un bande d'entraînement codée.



Lorsqu'un seul digit change, il n'y a plus de problème.

D'où une utilisation en automatisme des codes Gray

pour les compteurs et les codeurs.

Les codes Binaires Réfléchis sont des codes Grays symétriques

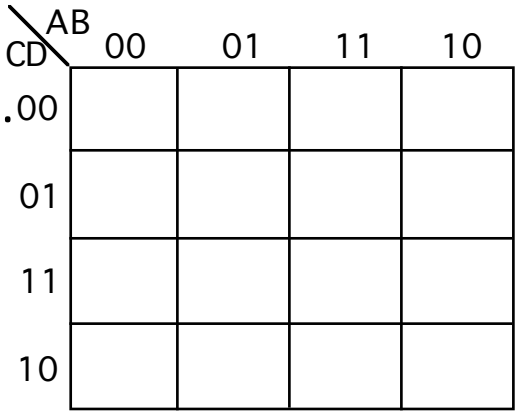
par rapport au milieu du code sauf le bit de poids Fort.

Représentation binaire des informations. Les codes minimum

7. LES CODES DE DONNÉES

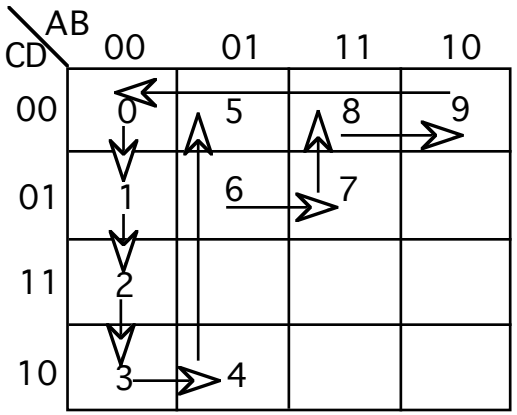
- **GÉNÉRATION DES CODES GRAY ET BINAIRES RÉFLÉCHIS**

On utilise des tableaux de Karnaugh. C'est un diagramme de WETCH ordonné c'est à dire que deux cases adjacentes ne sont modifiées que par une variable.

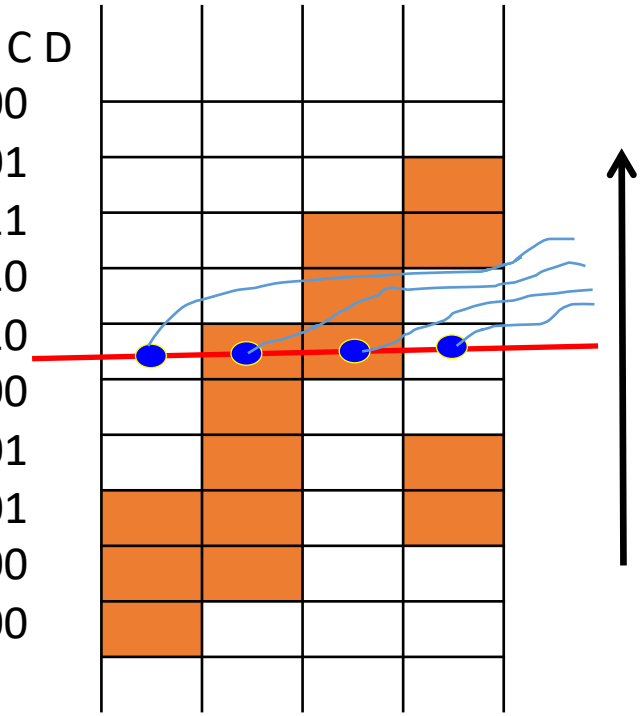


Les colonnes extrêmes sont adjacentes aussi entre elles de même que les lignes.

- **Exemple génération d'un code Gray de longueur 10.**



Décimal	A B C D
0	0000
1	0001
2	0011
3	0010
4	0110
5	0100
6	0101
7	1101
8	1100
9	1000



8. LES CODES INSTRUCTIONS

Les codes instructions sont des combinaisons binaires sur 1 ou plusieurs octets, correspondant à un ordre élémentaire exécutable par le processeur de la machine. C'est la plus petite action commandante par l'utilisateur au processeur. L'ensemble des combinaisons des codes instructions s'appelle le langage machine.

La construction du langage machine ne dépend en général pas d'une logique mathématique ou numérique particulière mais dépend de l'architecture interne de la machine et particulièrement de l'unité de commande. Par exemple seront regroupés avec les mêmes bits de poids forts, toutes les instructions ayant rapport au chargement de données mémoire vers l'unité centrale.

Cette dépendance entre les instructions (logiciel) et le matériel à pour conséquence que les codes instructions sont propres à chaque ordinateur et processeur. Les programmes écrits en langage évolué nécessite toujours une compilation pour être traduit en langage machine spécifique à la machine sur laquelle ils sont sensés s'exécuter. Le résultat de la compilation est la traduction du langage évolué en langage machine.

Les codes instructions sont en général implantés dans la zone instructions d'un programme. A la suite de chaque code on trouve des mots spécifiques sur 1 ou plusieurs octets, assujettis à une certaine règle et correspondant à des opérandes, à des adresses ou à des déplacements.